

Reflection Journal

My Observations About the Iris Dataset

Dataset Overview:

- Number of samples: 150
- Number of features: 4
- Number of classes: 3

Key Findings from the Visualization

- The iris setosa cluster forms a distinct, tight group with shorter sepal lengths and wider sepal widths, making it easily distinguishable from the other two species.
- Iris versicolor and Iris virginica overlap significantly in sepal dimensions, particularly around sepal length of 5.5 -6.5cm and sepal widths of 2.5-3.0cm.
- Iris setosa features a higher sepal width relative to its length. Conversely, as sepal length increases across versicolor and virginica, sepal width stays relatively constant or increases at a much lower rate

Questions for Further Investigation

- Would plotting petal length vs petal width provide a clearer linear boundary between Iris versicolor and Iris virginica than sepal measurements?
- Which machine learning would achieve higher accuracy when predicting species using all four features combined?

Reflection

By completing this exercise, I learned how seamlessly pandas DataFrames structure raw arrays into labelled tables and how NumPy simplifies quick array computations like mean and standard deviation. Additionally, visualizing data using Matplotlib revealed natural groupings and feature overlaps that were not obvious from looking at numeric tables alone. Before this lab, my mental model for structured data was strictly spreadsheet oriented. Coming from standard Excel tables, I initially found pandas DataFrames slightly non-intuitive specifically when transitioning from implicit row numbers to explicit index labels and handling multidimensional arrays. However, working through the indexing exercises triggered a key breakthrough: pandas indexing isn't just a row counter, but a high-performance hash-map key system that allows for relational alignment across complex datasets.

The contrast between native Python lists and vectorized NumPy arrays was another major point of growth. Previously, I viewed iteration through `for` loops as the default way to process sequential data. Watching vectorized operations compute mathematical metrics like mean and standard deviation across entire array columns instantaneously forced me to rethink computational efficiency. Seeing vectorization in action transformed NumPy from an abstract linear algebra library into an essential engine for scalable machine learning workflows.

My primary struggle during this lab was not in writing the Python code itself, but in managing plot boundaries, margins, and rendering layouts during document export[cite: 1]. When generating the sepal dimension scatter plots and class distribution bar charts using

Matplotlib, key elements like axis labels and visual legends were initially clipped or truncated during the PDF conversion process.

Initially, I tried manually tweaking figure dimensions, but this proved imprecise across different page aspect ratios. Furthermore, troubleshooting the Jupyter-to-PDF export pipeline highlighted a crucial lesson in software deployment: code execution is only half the battle; output presentation and static asset rendering require equal technical attention.